



Authentication

Hinzufügen Anmeldung Microsoft AD, Facebook, eigenes ASP.NET Identity
ApplicationUser Class, Gruppen, Authorization Attribute, Cors



Bekannte Gefahren für Software Systeme

- Cross-Site Scripting
 - Clientseitige Scripts, die Cookies und Session Tokens stehlen oder den DOM manipulieren können
- SQL Injection
 - Schadhafte SQL Anweisungen, die eine Datenbank zerstören können
- Cross-Site Request Forgery (CSRF bzw. XSRF)
 - User wird dazu veranlasst, schadhafte Requests abzusenden
- Open Redirects
 - URL Weiterleitungen auf gefährliche externe Websites z.B. nach Login

- + Open Redirects: Verwendung der LocalRedirect Helper Methode aus der Controller Base Class
- + CSRF: [ValidateAntiForgeryToken] Attribut auf Controller Action
- + SQL Injection: LINQ und Entity Framework
- + Cross-Site Scripting: Oauth 2.0



Sicherheit

- SSL
 - UseHttpsRedirection
 - UseHsts
- User Secret
- Cookie Authentifizierung
- JWT Bearer Tokens
 - *OAuth 2.0, Implicit Flow*
- CORS





SSL

- Require HTTPS für alle Requests
- Redirect aller HTTP Requests nach HTTPS
- Einstellen über Project Properties und Middleware

```
public void ConfigureServices(IServiceCollection services)
{
    services.Configure<MvcOptions>(options => {
        options.Filters.Add(new RequireHttpsAttribute()); });
}

public void Configure(IApplicationBuilder app)
{
    var options = new RewriteOptions().AddRedirectToHttps();
    app.UseRewriter(options);
}
```

<https://docs.microsoft.com/en-us/aspnet/core/security/enforcing-ssl>



SSL TLS HTTPS HSTS

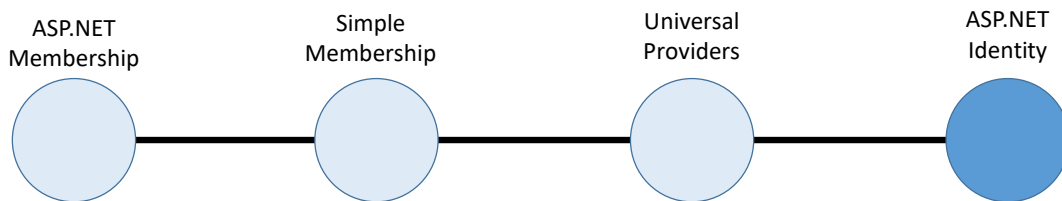
- Require HTTPS für alle Requests, SSL automatisch konfiguriert!
- Automatischer Redirect aller HTTP Requests nach HTTPS
- Einstellen über Project Properties und Middleware

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    if (env.IsDevelopment())
    {
        // Developer Middleware
    }
    else
    {
        app.UseHSTS();
    }
    app.UseHttpsRedirection();
}
```

<https://docs.microsoft.com/en-us/aspnet/core/security/enforcing-ssl>



ASP.NET Identity



Historischer Überblick über die Geschichte

Es gibt 4 Authentifizierungssysteme

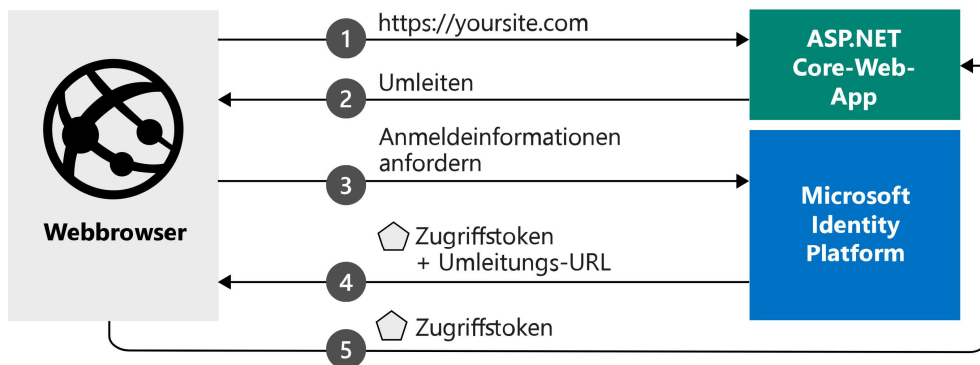
1. Kommt aus dem klassischen .Net Framework heraus.
 - Abstraktion aus SQL Datenbank heraus.
 - Authentication type: Individual Accounts.
 - Verwendet IdentityDbContext
2. Einfacher als Vorgänger -- flexibler -- besser integrierbar
3. ???
4. ASP.Net Identity hat was mit OAuth zu tun.
 - Es können beliebig viele Provider verwendet werden wie Azure Entra, Facebook etc.
 - Erweiterbarkeit und Integration z. B. mit OAuth, OpenID Connect, Claims-basierte Authentifizierung
 - Authentication type: Microsoft Identity Platform
 - Mit Azure Entra muss eine Auth App angelegt werden
 - Sollte aus Gründen der Kompatibilität von Teilnehmern NICHT nachgemacht werden

<https://stackoverflow.com/questions/21207246/asp-net-identity-vs-simple-membership-pros-and-cons>

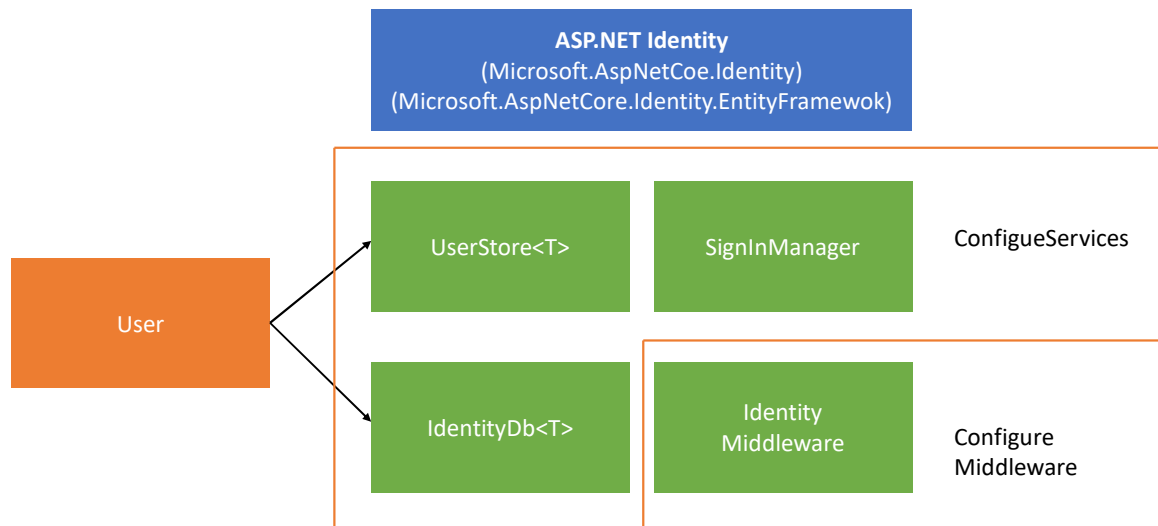
<https://www.hanselman.com/blog/introducing-systemwebproviders-aspnet-universal->

providers-for-session-membership-roles-and-user-profile-on-sql-compact-and-sql-azure

Fallbeispiel: Azure Entra ID (ehem. Azure AD)



Identity Framework Konfiguration





Authentifizierung vs. Autorisierung

- **Authentifizierung**

- Identity
 - Wer ist der Benutzer?
- Registrierter Benutzer
 - Lokale Membership-Datenbank
 - Third-Party Anbieter

- **Autorisierung**

- Permission
 - Was darf der Benutzer?
 - Welche Ressourcen darf der Benutzer verwenden?
- Authentifizierter Benutzer



Authentifizierung vs. Autorisierung

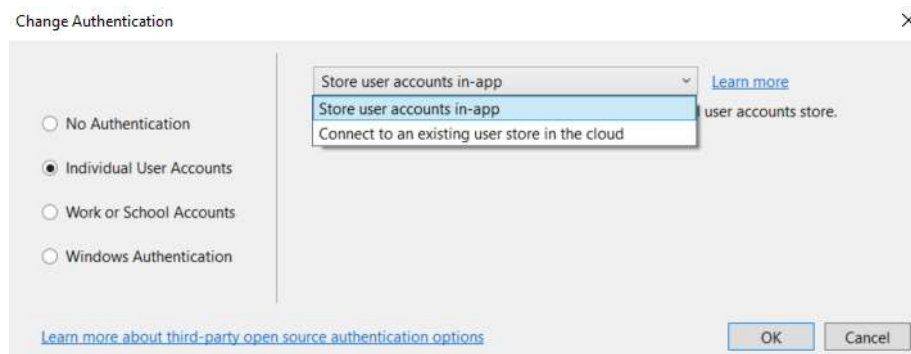
- Attribute [Authorize], [AllowAnonymous]
- Authentifizierung von z.B. POST, PUT, DELETE mittels Middleware

```
public void Configure(IApplicationBuilder app)
{
    app.UseAuthentication();
}
```



Identity Framework

- Möglichkeiten zur Absicherung mit ASP.NET Core Identity Framework





Identity Framework

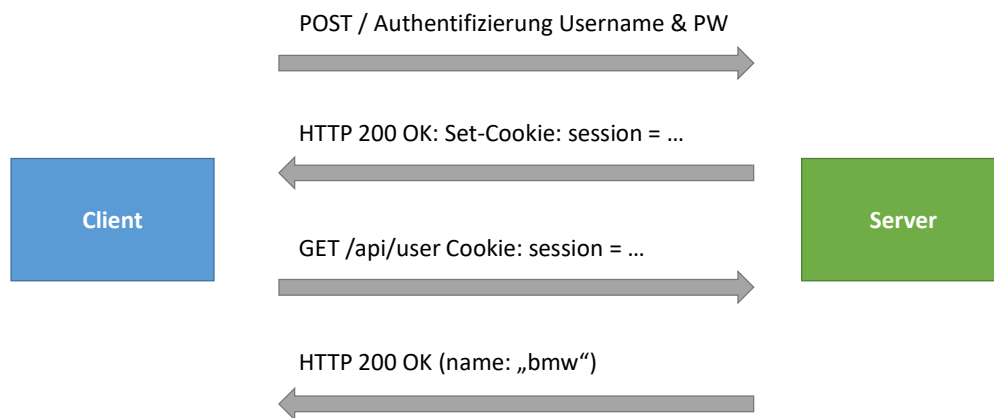
- Membership-System zur Verwaltung von Benutzern und zum Bereitstellen von Login Funktionalitäten in ASP.NET Core
 - Authentication & Authorization
 - Unterstützung externer Provider
 - Membership-basiert
 - SQL Server Support



<https://jonhilton.net/2017/05/03/login-authentication-asp-net-core-web-api-big-picture/>



Cookie Basierte Authentifizierung



<https://jonhilton.net/2017/10/07/a-simple-way-to-secure-your-.net-core-2.0-web-app/>



Middleware

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
        .AddCookie(CookieAuthenticationDefaults.AuthenticationScheme,
            options =>
            {
                options.LoginPath = new PathString("/auth/login");
                options.AccessDeniedPath = new PathString("/auth/denied");
            });
}
```

Claim Info über den Benutzer

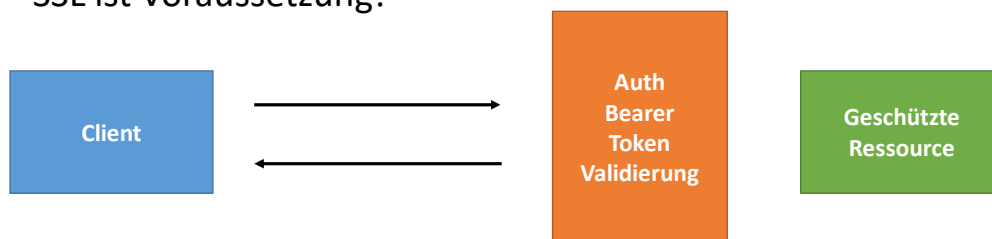
```
[HttpPost, ValidateAntiForgeryToken]
public async Task<ActionResult> Login(string returnUrl, string username, string password)
{
    if (username == "admin" && password == "password") {
        var claims = new List<Claim>
        {new Claim(ClaimTypes.Name, "admin", ClaimValueTypes.String, "https://ppedv.de")};
        var userIdentity = new ClaimsIdentity(claims, "SecureLogin");
        var userPrincipal = new ClaimsPrincipal(userIdentity);

        await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, userPrincipal,
            new AuthenticationProperties {
                ExpiresUtc = DateTime.UtcNow.AddMinutes(20),
                IsPersistent = false, AllowRefresh = false
            });

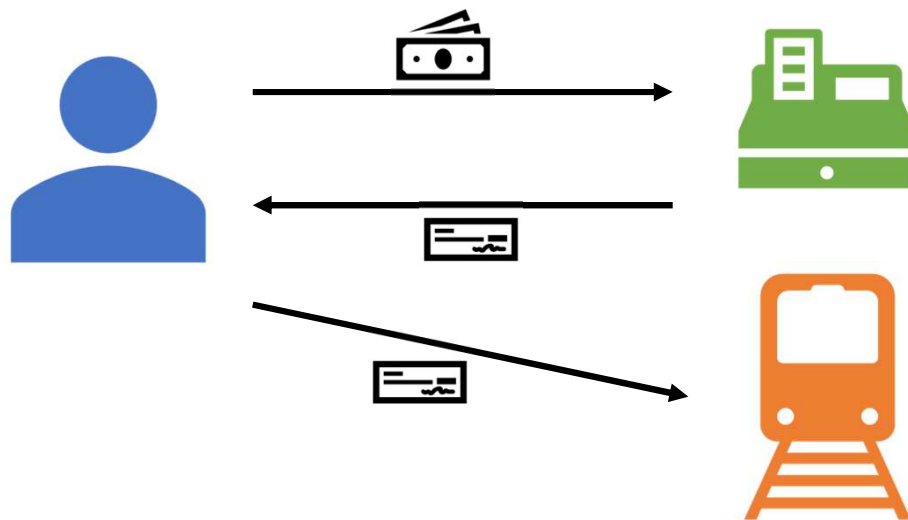
        return GoToReturnUrl(returnUrl);
    }
    return RedirectToAction(nameof(Denied));
}
```


OAuth 2.0 und JWT Bearer Tokens

- Speziell für APIs geeignet
- Requests können mittels OAuth 2.0 Bearer Token abgesichert werden
- Bearer Token werden über den Authorization Header eingebunden
- Middleware validiert Token und konvertiert diesen in Claims
- SSL ist Voraussetzung!



Beispiel: Fahrschein Authentifizierung & Autorisierung





OpenID Connect

- Spezifikation der Authentifizierungsschicht für OAuth 2.0
 - Identitätsprüfung
 - Profilinformationen
- Standard der OpenID Foundation
 - <http://openid.net/connect/>





OpenID Connect/OAuth 2.0 Flows (=Grants)

- **Authorization Code Flow**

- Client bezieht Authorisierungscode vom Server, Tokens werden serverseitig verwaltet
- Kommunikation zwischen Maschinen

- **Implicit Flow**

- Token Request ohne explizite Client Authentifizierung, Autorisierung durch Redirect URI
- Javascript Clients oder „traditionelle“ serverseitige Webanwendungen, Native Applikationen

- **Hybrid Flow**

- Mischform, Client bekommt Authorisierungscode zum Beziehen von Tokens
- Alternative für Web Anwendungen und native Apps => Verwendung von Refresh tokens

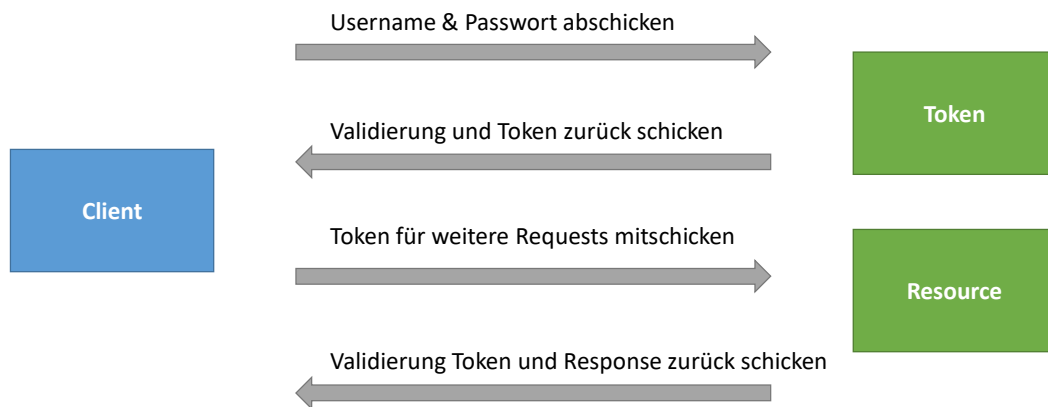
- Quelle:

<https://leastprivilege.com/2016/01/17/which-openid-connect-oauth-2-o-flow-is-the-right-one/>
<https://leastprivilege.com/2014/10/10/openid-connect-hybrid-flow-and-identityserver-v3/>

<https://www.scottbrady91.com/OpenID-Connect/OpenID-Connect-Flows>



Token Authentifizierung (Implicit Flow)



<https://jonhilton.net/2017/10/11/secure-your-asp.net-core-2.0-api-part-1---issuing-a-jwt/>



Token Authentifizierung

- JWT = JSON Web Token
- Standard für Tokens (RFC 7519)
 - Credentials
 - Claims
 - Weitere Informationen



- Beispiel JWT Bearer Token:

<https://jwt.io/>

Token Authentifizierung

- JWTs können ohne Third-Party Solution ausgestellt werden

```
[AllowAnonymous]
[HttpPost]
public IActionResult GetToken([FromBody] LoginDto user)
{
    if (user.Username == "admin" && user.Password == "password")
    {
        var claims = new[] { new Claim(ClaimTypes.Name, user.Username) };

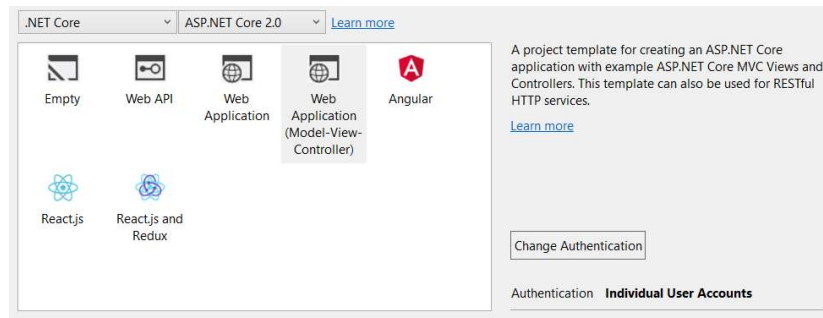
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["SecurityKey"]));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: "ppedv.de", audience: "ppedv.de", claims: claims,
            expires: DateTime.Now.AddMinutes(60),
            signingCredentials: credentials);

        return Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token) });
    }
    return BadRequest("Benutzername und Password nicht erkannt");
}
```

Externe Security Provider

- Autorisierung & Authentifizierung mittels *Google, Facebook, Microsoft & co.* möglich.



- <https://docs.microsoft.com/en-us/aspnet/core/security/authentication/social/>

Voraussetzung: Individual User Accounts

<https://samueleresca.net/2016/12/developing-token-authentication-using-asp-net-core/>



Externe Security Provider

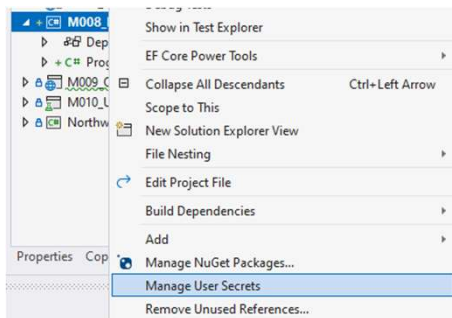
- Identity Server 4
- Security Framework für ASP.NET Core 2.0
- Basiert auf OpenID Connect und OAuth 2.0
- <http://identityserver.io/>
- <http://docs.identityserver.io/en/release/>
- <https://github.com/IdentityServer/IdentityServer4>





Secret Manager

- Lokal gespeichert
 - %APPDATA%\Microsoft\UserSecrets\



<https://docs.microsoft.com/en-us/aspnet/core/security/app-secrets?view=aspnetcore-2.2&tabs=windows>

<https://learn.microsoft.com/en-us/aspnet/core/security/app-secrets?view=aspnetcore-8.0&tabs=windows>

- Rechtsklick auf

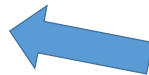
```
{  
  "AzureAD:ClientSecret": "lQ...bS4"  
}
```



CORS

- **Cross-Origin Ressource Sharing**

Origin: <http://ppedv.de>



Access-Control-Allow-Origin: <http://ppedv.de>



<https://www.c-sharpcorner.com/article/enabling-cors-in-asp-net-core-api-application/>



CORS

- Keine Domänen-übergreifenden AJAX Datenübertragung erlaubt
- Cross-Origin Ressource Sharing serverseitig in *Startup.cs* aktivieren!
- AddCors Middleware:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddCors();
}

public void Configure(IApplicationBuilder app)
{
    app.UseCors(x => x.AllowAnyHeader().AllowAnyMethod()
        .AllowAnyOrigin().AllowCredentials());
}
```